Intelligent Hospital Workflow Assistant

Deep Learning Project – Milestone 1

March 2026

1 Introduction

Healthcare institutions worldwide are under increasing operational pressure driven by growing patient demand, staff shortages, and the administrative complexity of managing day-to-day hospital workflows. Among the most critical yet frequently inefficient stages of hospital operations is the patient intake process — the sequence of steps through which a patient's condition is assessed, prioritized, matched to an appropriate physician, and assigned an appointment. In most institutions, this process remains largely manual, relying on administrative and nursing staff to simultaneously evaluate the severity of each case, identify the right department, and coordinate with physician schedules. The result is a workflow that is slow, inconsistent, and prone to human error, particularly in high-pressure clinical environments such as busy outpatient clinics and emergency departments. These inefficiencies have direct consequences for patient outcomes: urgent cases may not be identified and escalated quickly enough, while lower-priority patients may be assigned resources that are disproportionate to their needs.

This project proposes an Intelligent Hospital Workflow Assistant — an AI-powered system designed to automate the patient intake process from start to finish. Rather than replacing clinical staff, the system acts as an intelligent first point of contact: it receives the patient's reported symptoms, asks any follow-up questions needed to build a complete picture of their condition, determines the urgency of their case, identifies the most suitable doctor, schedules the nearest available appointment, and prepares a summary report for the physician before the consultation takes place. The goal is to reduce the administrative burden on hospital staff, minimize delays in patient routing, and ensure that the most urgent cases are always identified and prioritized correctly.

The real-world relevance of this system is significant. Delayed triage and misrouted appointments are among the leading causes of preventable adverse outcomes in outpatient

and emergency settings. By automating these steps through an intelligent system capable of reasoning about patient symptoms and acting on that reasoning without manual intervention, hospitals can serve more patients more efficiently, reduce waiting times, and allow clinical staff to focus their attention where it is most needed — on direct patient care.

2 Background & Related Work

2.1 Clinical Decision Support and Automated Triage

Clinical Decision Support Systems (CDSS) have long been used in healthcare to assist medical staff in making faster and more consistent decisions. Traditional systems relied on rigid, hand-crafted rules that could not adapt to complex or ambiguous patient presentations. More recent approaches have applied machine learning to automate triage classification from patient-reported symptoms, achieving accuracies of up to 92% on standard triage scales. These advances established the feasibility of replacing manual triage with intelligent automated systems.

2.2 Large Language Models in Healthcare

Large language models such as GPT-4 and Med-PaLM 2 have demonstrated strong performance on medical reasoning tasks, including symptom analysis, differential diagnosis, and clinical question answering. Their ability to process free-text symptom descriptions and generate structured clinical assessments makes them particularly well-suited to the patient intake use case. However, LLM outputs in medical contexts must always be treated as decision-support recommendations rather than autonomous diagnoses, and final clinical judgment remains with the physician.

2.3 Agentic AI and the ReAct Framework

Agentic AI refers to systems in which a language model does not simply respond to a single input, but autonomously reasons, plans, and takes actions across multiple steps to complete a goal. The ReAct framework formalizes this by structuring the model's behavior into a repeating Thought $\rightarrow$ Action $\rightarrow$ Observation loop, where the agent reasons before every action and updates its plan based on what it observes. This framework has been shown to significantly outperform single-shot LLM calls on multi-step tasks requiring real-world tool use.

2.4 LangChain as an Agent Framework

LangChain is an open-source framework designed specifically for building LLM-powered applications and agentic systems. It provides a production-ready implementation of the ReAct loop through its AgentExecutor, a tool registration interface that allows external functions to be called autonomously by the agent, and built-in conversation memory management. Its widespread adoption in both industry and research makes it the natural choice for implementing the agent at the core of the proposed system.

3 Proposed System Architecture

The Intelligent Hospital Workflow Assistant is composed of four principal layers: a React-based frontend, a FastAPI backend, a LangChain-powered AI agent, and a PostgreSQL database. These layers work together to deliver a fully automated patient intake experience, with the AI agent serving as the central decision-making component that coordinates all interactions between the system's parts.

3.1 System Components

The frontend is a lightweight React application that serves as the patient-facing interface. It consists of four screens: an email identification screen, a patient registration screen for new users, a symptom input and follow-up chat screen, and a confirmation screen displaying the appointment details. The frontend communicates with the backend exclusively through HTTP requests, and contains no business logic of its own.

The backend is built with FastAPI, a modern Python web framework chosen for its asynchronous request handling. This is particularly important in this system because LLM API calls introduce multi-second latency, and asynchronous handling ensures the server remains responsive to concurrent requests while waiting for the agent to complete its reasoning loop. The backend exposes a small set of endpoints that handle user identification, conversation turns, and result retrieval.

The AI agent is implemented using LangChain's AgentExecutor with a ReAct prompt. It serves as the reasoning and decision-making core of the system, receiving patient inputs, determining what information is still needed, invoking the appropriate tools, and ultimately producing a triage assessment, a doctor assignment, and a medical report. The agent is the only component that communicates with the LLM API.

The database is PostgreSQL, selected for its relational integrity and support for atomic writes — a critical property when booking appointment slots, as it prevents two patients

from claiming the same slot simultaneously. The database is accessed from the backend via SQLAlchemy, which provides a clean ORM layer over the raw SQL queries.

3.2 Agentic Components

The system satisfies all five components required of an agentic architecture.

The reasoning module is the LLM at the heart of the ReAct loop. At each step, it receives the full conversation history, the patient's profile, and the results of any previously invoked tools, and reasons over this information to decide what to do next — whether to ask a follow-up question, call a tool, or produce a final output.

The planning module is expressed through the agent's Thought steps. Before every action, the agent writes out its reasoning and determines the sequence of steps required to complete the intake workflow — for example, deciding to retrieve patient history before assessing symptoms, or recognizing that a priority 0 case requires immediate booking without further follow-up questions.

The tool integration layer consists of five LangChain-registered Python functions that the agent invokes autonomously: `get_patient_history`, which retrieves the patient's profile and past visits from the database; `find_available_doctor`, which queries the database for an active physician matching the required specialty; `book_appointment`, which writes the confirmed booking to the database and marks the slot as taken; `save_medical_report`, which stores the generated clinical summary and medication recommendations; and `send_confirmation_email`, which dispatches an appointment confirmation to the patient via SMTP.

The memory mechanism operates on two levels. Short-term memory is handled by LangChain's `ConversationBufferMemory`, which maintains the full dialogue history of the current session and injects it into every LLM call, allowing the agent to remember what the patient said in previous turns. Long-term memory is provided by the database: the patient's profile, chronic disease history, and records of past visits are retrieved at the start of each session and included in the agent's context, ensuring that the triage assessment is informed by the patient's complete medical background.

The execution pipeline is orchestrated by FastAPI and LangChain's `AgentExecutor`. Each message from the patient triggers a single call to the `AgentExecutor`, which internally runs the full ReAct loop — potentially spanning multiple reasoning steps and tool invocations — and returns only when the agent has produced a final response. The backend then forwards this response to the frontend and, where appropriate, persists the results to the database.

3.3 Database Schema

The database comprises six tables. The `users` table stores the patient's profile including email, age, and chronic disease history. The `doctors` table holds physician records including name, specialty, department, and availability status. The `doctor_slots` table stores pre-populated time slots for each doctor, each marked as booked or available. The `appointments` table records confirmed bookings, linking a patient to a doctor and slot, and storing the assigned priority level (0 for urgent through 3 for routine). The `medical_reports` table stores the LLM-generated clinical summary and medication recommendations for each visit, linked to both the patient and the appointment. The `conversation_history` table persists each turn of the agent's dialogue with the patient, organized by session, enabling future audit and review of the agent's reasoning process.

4 System Architecture Diagram

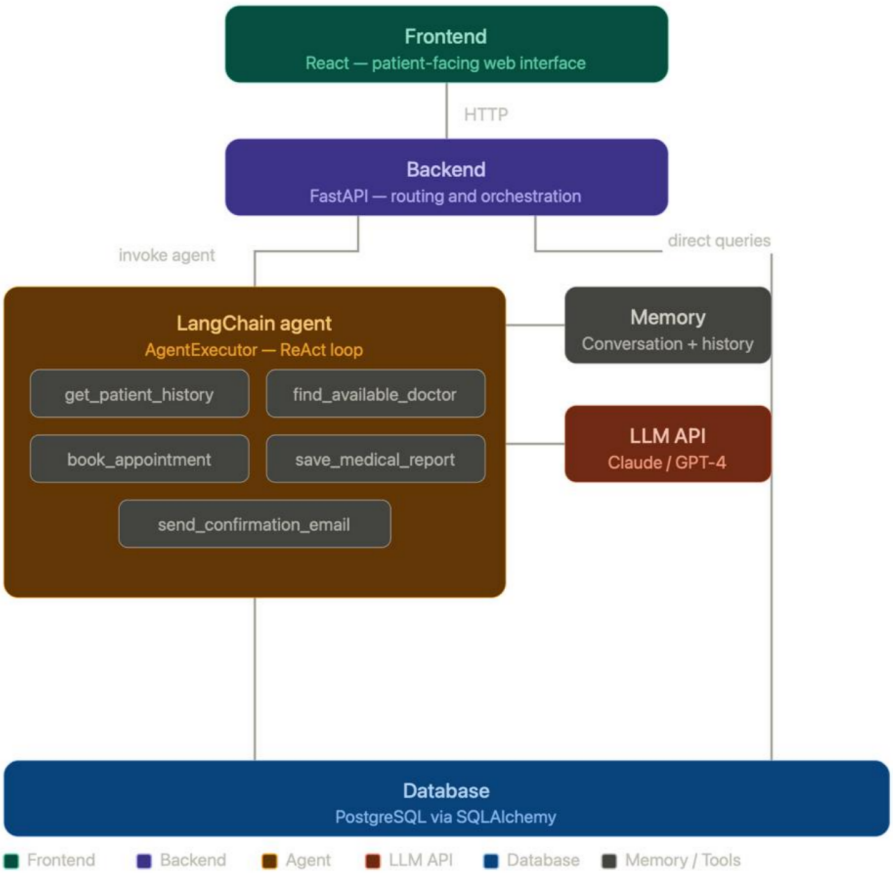

Figure 1: Conceptual overview of the Intelligent Hospital Workflow Assistant.

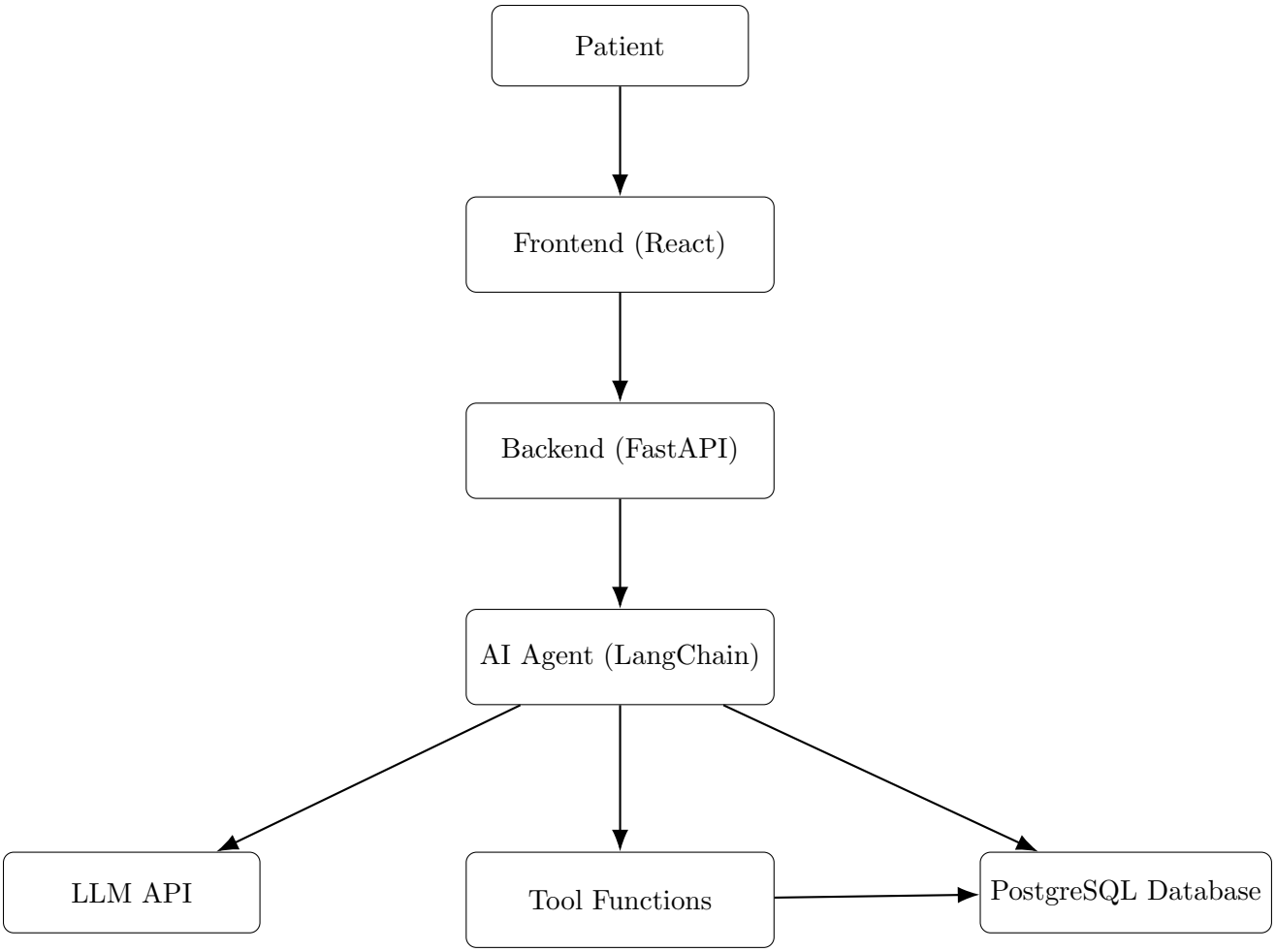

Figure 2: High-level architecture of the Intelligent Hospital Workflow Assistant.

5 Data Flow

This section describes how data moves through the Intelligent Hospital Workflow Assistant from the moment a patient begins interacting with the system until the appointment is scheduled and the clinical report is stored. The system processes patient input through a sequence of components that coordinate data exchange between the frontend, backend, AI agent, and database.

5.1 User Identification

The interaction begins when the patient enters their email address through the frontend interface. The frontend sends this information to the backend using an HTTP request. The backend then queries the database to determine whether the user already exists in the system.

If the user is already registered, the backend retrieves the patient's profile information, including age, chronic disease history, and records of previous visits. If the email address does not exist in the database, the system requests additional information such as age and chronic disease history in order to create a new user profile.

5.2 Symptom Submission

Once the user has been identified or registered, the patient submits their current symptoms through the frontend interface. The frontend sends this information to the backend, which forwards it to the AI agent together with the patient's profile and the full conversation history of the current session.

This information forms the context that the agent uses to analyze the patient's condition and determine the next step in the intake process.

5.3 Agent Reasoning and Information Gathering

The AI agent processes the received data using the ReAct reasoning loop. Based on the provided symptoms and patient profile, the agent determines whether sufficient information is available to perform a triage assessment.

If the information is incomplete, the agent generates follow-up questions that are sent back through the backend to the frontend, prompting the patient to provide additional details such as symptom duration, severity, or related conditions. The patient's responses are then fed back into the reasoning process.

5.4 Doctor Assignment and Appointment Booking

After collecting sufficient information, the agent determines the urgency level of the case and identifies the most appropriate medical specialty. The agent then invokes the relevant tool to query the database for an available doctor within that specialty.

Once a suitable doctor and time slot are found, the agent triggers the appointment booking process. The backend writes the confirmed appointment to the database, ensuring that the selected slot is marked as booked to prevent scheduling conflicts.

5.5 Report Generation and Output

Following the appointment booking, the AI agent generates a structured medical report summarizing the patient's symptoms, the triage assessment, and any suggested medication

recommendations. This report is stored in the database and associated with the patient's appointment record.

Finally, the backend returns the appointment details to the frontend, which displays a confirmation screen to the patient. Optionally, the system may also send an email confirmation containing the appointment information and the assigned doctor's details.

5.6 Patient Intake Flow Diagram

Refer to Figure 5

6 Deployment Design

The proposed system will be deployed locally during the development phase. The application will run on a local server environment where all system components operate on the same machine.

The deployment setup consists of four main components. The frontend will run on a local development server that provides the user interface through the browser. The backend will run on a FastAPI server responsible for handling API requests, coordinating the system logic, and managing communication between components. A PostgreSQL database server will run locally to store persistent data such as user profiles, appointments, medical reports, and conversation history. In addition, the AI agent will run within the backend environment and will be responsible for processing patient inputs, reasoning over the available data, and invoking the necessary tools to complete the patient intake workflow.

These components communicate through local network interfaces. The frontend interacts with the backend through HTTP requests, while the backend communicates with the PostgreSQL database to retrieve and store system data. The AI agent operates as part of the backend service and interacts with the external language model through APIs to perform reasoning and decision-making tasks.

6.1 Deployment Diagrams

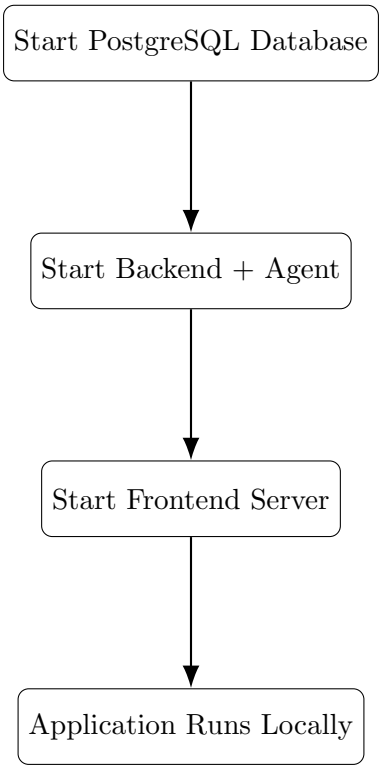

Figure 3: Local deployment architecture of the system.

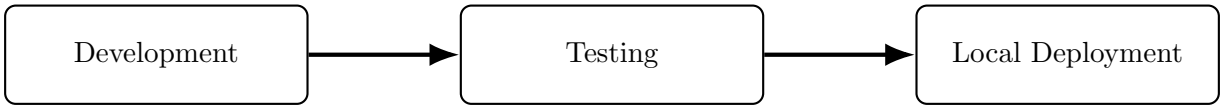

Figure 4: Development and testing workflow before local deployment.

7 Model Comparison Table

Table 1: Comparison of Large Language Models Considered for the Intelligent Hospital Workflow Assistant

Model	Context Window	Medical Reasoning	Structured Output	API Access	LangChain Support	Cost
GPT-4o (OpenAI)	128K tokens	Excellent	Native JSON mode	Yes	Native	High
GPT-3.5 Turbo (OpenAI)	16K tokens	Moderate	JSON mode	Yes	Native	Low
Claude 3.5 Sonnet (Anthropic)	200K tokens	Excellent	Reliable	Yes	Native	Moderate
Gemini 1.5 Pro (Google)	1M tokens	Good	Reliable	Yes	Native	Moderate
Llama 3 70B (Meta)	8K tokens	Good	Prompt-dependent	Self-hosted	Via Ollama	Free

Medical reasoning ratings are based on comparative benchmark evaluations reported in Nori et al. (2023) , Singhal et al. (2023) , and Zheng et al. (2024) . Structured output and LangChain compatibility ratings are based on the official LangChain documentation .

8 Section 2: Interesting Insight & Unique Learning

8.1 The Ethical Tension Between Automation and Clinical Responsibility

One of the most thought-provoking discoveries during the design of this system was the realization that automating clinical triage introduces a fundamental ethical question that technology alone cannot resolve: if an AI agent assigns the wrong priority level to a patient and that patient is harmed as a result, who bears responsibility? The hospital? The developers? The LLM provider? This question is not hypothetical. In several documented cases of AI-assisted clinical decision support, systems that performed well on benchmark datasets failed in deployment due to distribution shift — the real-world patient population behaved differently from the training data. More critically, patients tend to over-trust automated systems, particularly when they are presented through a polished interface, and may not escalate their condition independently if the system reassures them incorrectly. The design implication for our system is significant. Every output of the agent — the triage level, the doctor assignment, the medication recommendations — must be explicitly framed as a preliminary recommendation pending physician review, not a clinical decision. The doctor's intake report must clearly indicate that it was AI-generated. This is not merely a disclaimer — it is a structural requirement that determines how the system should be built and how its outputs should be displayed. Designing for human oversight is not an afterthought; it is a core architectural principle.

8.2 Why ReAct Is Not As Reliable As It Sounds

The ReAct framework is elegant in theory: the agent reasons, acts, observes, and repeats until it reaches a confident conclusion. In practice, however, the pattern has well-documented failure modes that became apparent when we studied the literature in depth. The most significant is reasoning **hallucination** — the agent's Thought step is generated by the same LLM that produces the final answer, meaning the reasoning itself can be flawed or fabricated. ReAct agents can enter repetitive loops, convince themselves they have sufficient information when they do not, or misinterpret a tool's output and proceed with an incorrect plan. In a medical context, this is particularly dangerous: an agent that incorrectly interprets a patient's symptom description and confidently assigns Priority 3 to what is actually a Priority 0 case is not just making a technical error — it is potentially contributing to a life-threatening delay. A second failure mode is **tool call errors** — the agent may invoke a tool with malformed or incomplete arguments, receive an error, and either retry incorrectly or skip the step entirely. LangChain's AgentExecutor provides some built-in error handling, but it does not guarantee correct recovery in all cases. This insight directly informed our decision to implement explicit validation on all tool inputs and to log every tool call and its result to the database, creating an auditable trail that can be reviewed if something goes wrong.

8.3 LLM Context Window Limits and Long Conversations

The practical consequence is that a patient with a long history could have their earlier responses — potentially the most clinically relevant ones — silently dropped from the context before the agent makes its final assessment. This is a non-obvious failure mode that would be very difficult to detect in testing but could have serious consequences in deployment. Our insight here is that **ConversationBufferMemory** is not appropriate for production medical applications without modification. A more robust approach would be to use **LangChain's ConversationSummaryMemory**, which periodically summarizes older parts of the conversation rather than retaining them verbatim, keeping the context within safe limits while preserving the semantic content. This represents a meaningful architectural improvement that we intend to evaluate during the implementation phase.

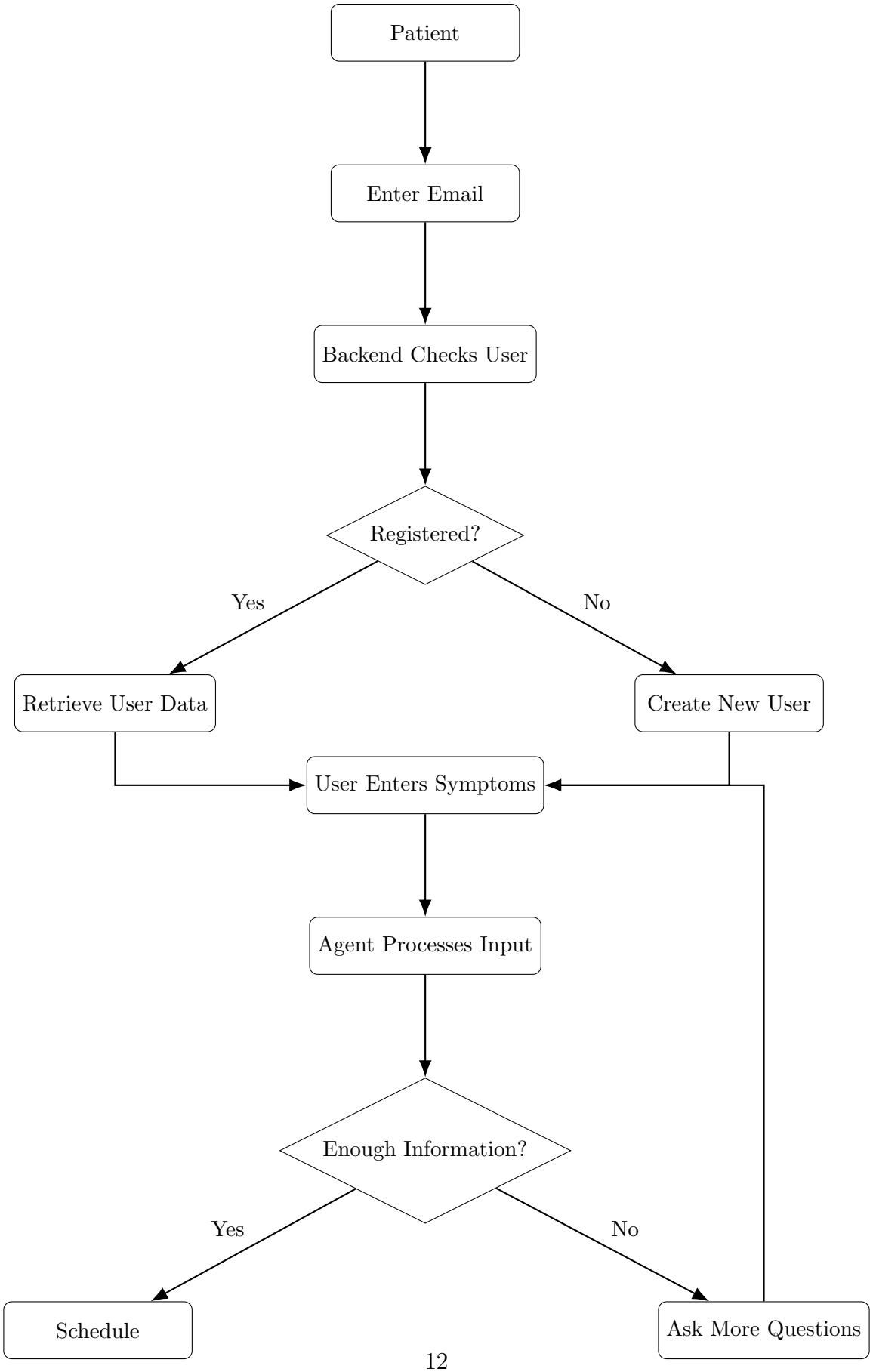

Figure 5: Patient intake data flow in the Intelligent Hospital Workflow Assistant.